agentia-ansible

Copado releases, run the way the rest of your platform already runs.

An Ansible collection, an Event-Driven Ansible source, an Agentia CLI plugin command, an MCP server and an Agent Skill, all built on the Copado Agentia Headless CLI and verified on a real Copado org.

Chaman Swami

github.com/chamanswami555/agentia-ansible

THE PROBLEM

One change ticket, one automated run, and a Salesforce step still done by hand

- A Salesforce change rarely ships alone: the same ticket carries a database migration, an infrastructure change, a feature flag. Platform teams already run those from Ansible Automation Platform: inventories, approval nodes, schedules, RBAC, a job history that doubles as evidence.
- Copado releases sit outside that: someone presses Submit, waits for jobs, pastes screenshots into the change ticket.
- Validation, smoke tests, approval and deployment are four manual steps. A failed job at night waits until morning.

Agentia Headless changes the input

Every step is now a CLI command with JSON output.

But a CLI is not a building block

Each team would still write its own polling, parsing, error handling and approval logic.

One collection, five ways in

Piece	What it is
Collection `chamanswami555.agentia`	12 idempotent modules with check and diff mode, Copado transaction ids, the CLI's own error messages, a real per-call timeout. 3 playbooks: release window, nightly validation, remediation of a failed job.
Event-Driven Ansible	Source `agentia_jobs` polls job executions and emits status changes; rulebook `copado_release` runs the remediation playbook on Error or Failed.
Agentia CLI plugin	`agentia ansible scaffold` reads the active story through the plugin SDK and writes the playbook; `agentia ansible run` runs it with `--check`, `--approve`, `--json`.
MCP server	Five governed tools for AI agents: list, run release, run playbook, list evidence, release status. Without approval the run stops at the gate.
Agent Skill	Teaches a coding agent the module order, the real Copado shapes and the rules for safe playbooks.

THE RELEASE WINDOW

Validate, prove, approve, ship, with evidence at every step

1. Health check
Copado governance checks before anything moves.

2. Validate
Each story: `work submit`, wait for the promote job, then for the check-only deployment the promotion record names.

3. Smoke suite
Robotic Testing run, optional.

4. Approval gate
A typed yes, or `approve: true` from an AWX node, MCP or CI. A plain pause is skipped without a terminal; this one fails closed.

5. Copado Submit
`work submit --done` per story: promote job, deployment job, both waited for. Or merge_and_deploy on an existing promotion.

6. Evidence
validations, deployment, health checks before and after, as JSON with transaction ids.

Nightly validation runs steps 1 to 3 for every story in flight and fails loudly. Remediation reads a failed job and its log, asks Copado AI for a diagnosis, writes an incident, and can resume the job.

Every surface Agentia offers, used for what it is good at

Surface	How agentia-ansible uses it
CLI with `--json`	Every module runs a public command, parses the `{result, status, transactionId}` envelope, tolerates the warning line a linked plugin prints, returns the transaction id, fails with Copado's own message.
Plugin SDK and generator	Generated with `create-agentia-plugin`; `scaffold` reads the active story from the host API; `run` finds the collection on its own and reports a stop at the gate. Declares its permissions in package.json.
MCP	Governed workflows instead of raw commands: an agent can validate, read evidence and release only through the playbooks, with the approval gate in place.
Agent Skills	A skill with the facts learned on the real system, so a coding agent writes playbooks that match what Copado actually does.

LIVE PROOF

Everything ran against a real Copado org

Run on the Copado Playground, 14 to 16 October 2026	Result
Module smoke test: auth, health check, story create, idempotent re-run, update, delete, job reads (on CLI beta.5, beta.6 and rc.0)	8 ok, 3 changed, 0 failed
Release window through the plugin: US-0000024 from INT to UAT (validation P00005, Copado Submit P00006, jobs JE-010 to JE-013)	17 ok, 7 changed, 0 failed
Release window with the validation-deployment wait: US-0000026 from Dev1 to INT (P00007, P00008, jobs JE-015 to JE-018)	21 ok, 7 changed, 0 failed
Run without approval and without a terminal: validation P00010 ran, the gate stopped the run	Nothing deployed, exit 2
Event source inside ansible-rulebook; the shipped rulebook during a release	11 job events; 4 status changes, rule fired on each
MCP server over stdio in check mode; local quality gate through `work test` and a live validation submit	14 ok, 0 failed; gate ran, exit 0

Nobody watches a console; nobody deploys without a decision

A job fails at 02:00

The source sees the status change, the rule fires, the remediation playbook reads the job and its log, asks Copado AI for a diagnosis, writes the incident file, and resumes the job if the team allows it. Proved end to end offline and live.

An agent asks for a release

It gets five tools. `run_release_playbook` without `approve: true` stops at the gate and returns `stopped_at_approval`. There is a test that runs without approval and checks that nothing was deployed.

```
$ agentia ansible scaffold --stories US-0000026
Wrote release-window.yml
  stories:    US-0000026
  deploy:     story mode (Copado Submit per story)

$ agentia ansible run --playbook release-window.yml --check
Playbook finished: {"ok":14,"changed":7,"failed":0}

$ agentia ansible run --playbook release-window.yml --approve
Stories US-0000026 released (story mode).
Deployment jobs: ['Successful'].
```

ENGINEERING QUALITY

Tests that fail when the code they protect is removed

75

unit tests, 96 percent coverage, 90 percent floor in CI

24 + 5

ansible-test sanity tests and integration targets, on ansible-core 2.17 and 2.18, plus ansible-lint at the production profile

7 + 1

offline demo steps and the rulebook demo against a fake CLI; 45 real recordings, scrubbed, back the shape tests

6 + 13

MCP and plugin tests, including the release end to end, the gate without approval and both plugin commands

- Mutation checks: the approval gate, the fake call log, the story lookup and the interval bug each have a test that goes red when the fix is removed.
- Fixtures recorded from the real org and scrubbed; their README lists the origin of every file. The live checks re-ran on CLI 1.0.0-beta.6 and 1.0.0-rc.0 after each release.
- CI on every push, on ansible-core 2.17 and 2.18: lint, unit tests, sanity, integration, both demos, MCP and plugin tests, evidence kept as an artifact.

WHY NOT JUST...

The questions we asked ourselves first

Objection	Answer
Call the CLI from a shell task?	No change reporting, no dry run, and polling, parsing, pagination and errors rewritten in every playbook. The modules are the tested layer.
Use Copado's own automations?	They run inside Copado. The change window spans Salesforce, databases, infrastructure and flags, and runs from Ansible with its approvals, RBAC and schedules.
Use a CI pipeline?	You can; our CI does. Ansible adds approval nodes, inventories, credentials, schedules and event-driven remediation.
Use the Agentia MCP server?	194 raw tools versus five governed workflows with an approval gate and an evidence folder.
What does it not do?	No rollback (the CLI has none), no Windows control node, Robotic Testing and Copado AI verified against recorded answers only. Listed, not hidden.

NEXT

Ready to install, ready to extend

- Repository: **github.com/chamanswami555/agentia-ansible** (MIT)
- Collection: `ansible-galaxy collection install chamanswami555.agentia` after publication; builds and installs from the repository today
- Plugin: `agentia plugins link plugin`
- MCP: `node mcp/server.mjs`, client configuration in mcp/README.md
- Rollback module as soon as the CLI exposes the operation
- AWX job template and workflow with an approval node
- Ephemeral environment per user story (scratch org, credentials, pipeline connection) as a playbook
- Windows gate script and a Galaxy release cadence that follows the CLI betas

Copado releases, run the way the rest of your platform already runs.